

Dédicace

Je voudrais tout d'abord remercier mon Dieu qui m'a aidé à chaque pas pour que je puisse réaliser mes ambitions. Je dédie ce mémoire de fin d'études :

A mon très cher père Chokri : Pour vos immenses sacrifices, votre courage et surtout votre patience et votre compréhension. Grâce à votre bienveillance, à votre encouragement et à votre générosité, j'ai pu terminer mes études dans L'enthousiasme. Je ferai de mon mieux pour rester un sujet de fierté à vos yeux avec l'espoir de ne jamais vous décevoir. Que ce travail puisse être le résultat de vos efforts. Que Dieu le tout puissant soit à vos côtés et vous accorde une meilleure santé.

Au symbole de douceur et d'affection ma mère Chiraz : Pour l'amour dont tu m'as toujours entouré, pour le sacrifice et le dévouement dont tu as toujours fait preuve, pour l'encouragement sans limites que tu ne cesses de manifester. Aucun mot, aucune phrase ne peut exprimer mes sentiments profonds d'amour, de respect et de reconnaissance. Que ce modeste travail soit un début de mes récompenses envers toi.

A Toute ma Famille : Qui a été toujours pour moi la garante d'une existence paisible et d'un avenir radieux.

A mes chers amis : Merci pour l'amitié qui nous a unies et pour tous les merveilleux moments et les souvenirs partagés. ...

Aymen

Avant de rédiger ce rapport, nous tenons à remercier toutes les personnes ayant contribué au bon déroulement de notre stage de projet de fin d'études.

Au terme de ce travail nous tenons à remercier vivement notre encadreuse Mme. **Sondos Zina** pour son encadrement, sa disponibilité et ses précieux conseils qui nous ont prodigué pendant la durée de notre stage de fin d'études au sein de la **banque centrale de tunisie**.

Par la suite, nous tenons également à exprimer notre profond respect et à remercier infiniment mon encadrante à ESPRIT Mme. **Manel Khamassi** pour son aide utile et précieuse, son excellent encadrement et ses encouragements qui nous ont motivés à réaliser ce travail.

On adresse nos sincères remerciements et tous nos respects à tous les membres du jury

qui ont bien voulu accepter de juger notre rapport de projet de fin d'études.

On tient à remercier aussi tous le personnel de la **banque centrale de tunisie** qui nous a aidés et nous a encouragés à réaliser ce rapport et pour l'ambiance amicale tout le long de ce projet.

Il est indispensable de ne pas rater cette occasion pour exprimer notre gratitude et notre respect à **nos enseignants à ESPRIT** pour leur collaboration et soutien durant nos études.

Abréviations

API	Application Programming Interface
AHP	Analytic Hierarchy Process
AST	Abstract Syntax Tree
AUC	Area Under the Curve
CSV	Comma-Separated Values
DPG	Digital Public Goods
DPI	Digital Public Infrastructure
G2P	Government-to-Person
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MLflow	Machine Learning Flow
NLP	Natural Language Processing
ORM	Object Relational Mapping
PIAF	Pipeline Intégré d'Analyse de Fraude
PMT	Proxy Means Test
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SQL	Structured Query Language
UML	Unified Modeling Language
XAI	Explainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting

Table des matières

Table des figures

Table des tableaux

Introduction Générale

Dans un monde où **1,3 milliard de personnes** vivent encore sous le seuil de pauvreté, la protection sociale n'est plus un luxe, c'est un impératif de survie. Chaque jour, des millions de familles dans les pays émergents et en développement dépendent d'un transfert monétaire pour nourrir leurs enfants, payer une consultation médicale ou maintenir un enfant à l'école. Ces programmes de transferts monétaires directs, désignés sous le terme **Government-to-Person (G2P)**, constituent aujourd'hui le mécanisme le plus efficace pour réduire la pauvreté et les inégalités dans les pays les plus vulnérables.

Pour rendre ces programmes possibles à grande échelle, les gouvernements se tournent vers un concept transformateur : **les Infrastructures Publiques Numériques (DPI)**.

Les DPI désignent l'ensemble des systèmes numériques partagés, identification biométrique, registres sociaux centralisés, systèmes de paiement interopérables qui permettent à un État de fournir des services publics de manière efficiente, transparente et inclusive.

Pour concrétiser cette vision, les gouvernements s'appuient sur **les Biens Publics Numériques ou DPG** : des logiciels gratuits et open-source que n'importe quel pays peut adopter et adapter à son contexte local.

OpenG2P est l'un de ces DPG. Une plateforme à code source ouvert qui permet de gérer l'ensemble du processus : identifier les bénéficiaires, organiser les programmes d'aide, et suivre les paiements. Déjà déployée au Kenya, au Malawi et en Sierra Leone, OpenG2P offre aux pays à budget limité des outils performants sans dépendance à un éditeur privé.

Mais cette promesse porte en elle une menace qui risque de la compromettre.

Les programmes G2P font face à un adversaire silencieux : **la fraude**. Les tactiques sont multiples : multi-inscription sous des identités différentes, ménages fictifs, comptes bancaires partagés entre individus frauduleux, manipulation de paiements. Selon la Banque mondiale, **jusqu'à 27 % du budget** destiné à la protection sociale est perdu à cause de ces fuites.

Le problème est aggravé par le fait que les contrôles actuels interviennent trop tard. Les audits manuels sont lents, coûteux, et ne couvrent qu'un échantillon limité. Les solutions commerciales, quant à elles, sont coûteuses et incompatibles avec la philosophie open-source des DPG.

C'est dans ce cadre que notre stage effectué au sein d'EY Tunisia s'inscrit : concevoir un moteur de détection libre, proactif et explicable, intégré à OpenG2P, pour protéger les budgets de protection sociale avant même le versement.

Chapitre 1

Contexte du proojet

1.1 Introduction

Ce chapitre expose le contexte général des travaux menés, nous allons présenter le contexte global du projet, en débutant par l'introduction de l'organisme d'accueil qui est « EY Tunisie », décrire ses activités et ses différentes équipes. Par la suite, nous procédons à la description de notre problématique qui a mené au développement de ce projet. Enfin, nous expliquons la méthodologie choisie pour accomplir ce travail.

1.2 Présentation de l'organisme d'accueil



FIGURE 1.1 – Logo d'EY

Fondé en 1849, Ernst and Young est un cabinet d'audit financier et de conseil multinational implanté dans plus de 152 pays organisés en trois régions : Amérique ; Europe, Moyen Orient, Inde et Afrique (EMEIA) ; Asie-Pacifique. EY fait partie des « Big Four », soit les quatre plus grands groupes d'audit à l'échelle mondiale. EY est une organisation de services professionnels approfondie avec plus de 700 bureaux et 350 000 associés et collaborateurs dans le monde entier. Toutes les prestations offertes par le cabinet EY ont enregistré une progression en 2022, ce qui a conduit à une hausse du chiffre d'affaires global d'environ 24,523 milliards de dollars américains ce qui correspond à 7.3 EY est une entreprise de premier plan à l'échelle mondiale évoluant dans le domaine d'assurance, de fiscalité, de transaction, et de conseil. EY bénéficie d'une équipe mondiale en expansion constante

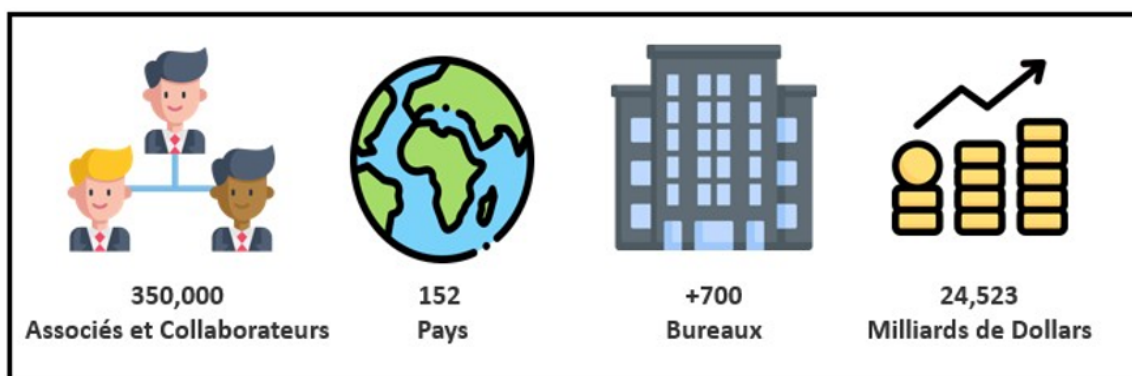


FIGURE 1.2 – EY Global

EY est une entreprise de premier plan à l'échelle mondiale évoluant dans le domaine d'assurance, de fiscalité, de transaction, et de conseil. EY bénéficie d'une équipe mondiale en expansion constante.

1.2.1 AMC EY Tunisie

Établie sur le territoire depuis 1987, AMC EY Tunisie se positionne comme l'une des pionnières dans le domaine des services professionnels du pays. Elle fait partie de la région EMEIA et compte plus de 1000 employés avec plus de 900 consultants en service de conseil. Grâce à plus de 620 missions réalisées annuellement, l'entreprise a atteint un chiffre d'affaires de 50 Millions de dinars tunisiens en 2022 avec plus de 50% réalisées auprès de clients internationaux.



FIGURE 1.3 – AMC EY Tunisie

EY Tunisie intervient dans différents secteurs économiques en Tunisie, notamment dans le secteur des technologies de l'information et de la communication, méritant la confiance tant du secteur public local (ministères, agences de développement local) que du secteur privé grâce à la proposition de solutions adaptées et pertinentes.

1.2.2 Services proposés par EY Tunisie

EY Tunisie structure son offre autour de quatre grands métiers complémentaires qui permettent à l'entreprise de répondre aux besoins variés de ses clients et de les accompagner dans leur développement et leur transformation, résumés dans le tableau suivant :

Service	Description
Conseil	EY aide ses clients à créer de la valeur à long terme pour toutes les parties prenantes. En s'appuyant sur les données et la technologie, ses services et ses solutions orientent les clients et les aident à prendre des décisions sur le plan stratégique et opérationnel afin de parvenir favorablement à leurs attentes.
Audit	EY fournit des renseignements et cherche à offrir aux clients (investisseurs) la meilleure gestion et l'adéquate structuration de ses rapports et de ses opérations dans le but d'améliorer voire de valoriser leurs tâches et d'accroître leurs opportunités envers les autres entreprises.
Fiscalité et droit (Tax)	EY possède une expertise en matière d'impôt sur les sociétés, d'impôt international, d'impôt sur les transactions et de ressources humaines, de conformité et de déclaration, et d'impôt statutaire.
Stratégies et Transactions	EY aide ses clients à surmonter la crise et à stabiliser leurs activités à court terme. EY aide également les entreprises à surmonter les ralentissements et à se préparer à la reprise, ou à regarder au-delà pour permettre la transformation par le biais de la stratégie d'entreprise, des transactions, des fusions et acquisitions et des désinvestissements.

TABLEAU 1.1 – Services professionnels d'EY Tunisie

1.2.3 Présentation du département Technology Transformation

Le département « **Technology Transformation** » qui fait partie du service line Conseil occupe une position centrale dans la transformation d'entreprise et propose des possibilités d'optimisation des rendements, de supervision des risques et de promotion de l'innovation des entreprises.

En s'appuyant sur une gamme diversifiée de projets technologiques, la division « **Technology Transformation** » offre à ses clients la possibilité d'améliorer leurs pratiques et d'acquérir un avantage compétitif. L'équipe offre cinq (5) principales activités pour cinq (5) secteurs intersectoriels, comme l'illustre la figure ci-dessous.

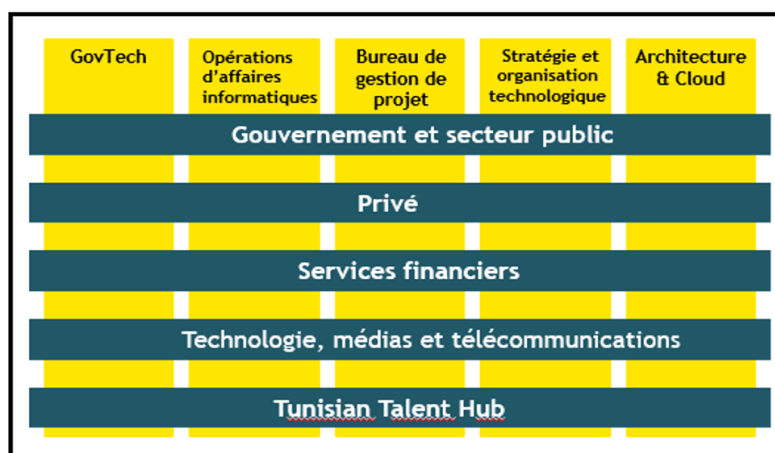


FIGURE 1.4 – Les activités de la division « Technology Transformation »

1.3 Présentation du projet

1.3.1 Cadre du projet

Notre projet s'inscrit dans le cadre de la préparation du stage de fin d'études pour l'obtention du Diplôme National d'Ingénieur en Informatique. Notre mission consiste à concevoir et développer un moteur intelligent de détection des risques et fraudes intégré à la plateforme de protection sociale OpenG2P, réalisé au sein du cabinet EY Tunisia.

1.3.2 Contexte du projet

Le projet s'inscrit dans l'axe de modernisation des services publics, où EY accompagne les organismes gouvernementaux dans l'adoption de solutions technologiques modernes. Notre mission porte sur la plateforme OpenG2P, un logiciel libre utilisé par plusieurs pays pour gérer leurs programmes d'aide sociale : identifier les bénéficiaires, organiser les versements, et suivre les paiements.

Afin de bien cerner les enjeux de ce projet, nous avons commencé par étudier en profondeur le fonctionnement d'OpenG2P et de son écosystème. Les observations issues de cette étude nous ont permis d'identifier des lacunes critiques en matière de lutte contre la fraude, que nous détaillons dans les sections suivantes.

1.3.3 Problématique

Les programmes de protection sociale gèrent chaque année des flux financiers considérables destinés aux populations vulnérables. Cependant, selon la Banque mondiale, **jusqu'à 27% de ce budget** est perdu à cause de la fraude et des erreurs de ciblage. Les principaux problèmes identifiés sont les suivants :

- **La fraude prend des formes variées et évolutives** : faux bénéficiaires, ménages fictifs, identités partagées, manipulation de paiements.
- **Les contrôles actuels interviennent après le versement** : quand l'argent a déjà quitté les caisses de l'État.
- **Les vérifications manuelles** sont lentes, coûteuses, et ne couvrent qu'un échantillon limité.
- **Les règles de contrôle figées** ne détectent que les fraudes connues et ne s'adaptent pas aux nouvelles tactiques.
- **Les solutions commerciales** existantes sont **coûteuses, opaques, et incompatibles** avec la philosophie open-source d'OpenG2P.

Il devient donc indispensable de disposer d'une solution intelligente, proactive et explicable, capable de détecter la fraude **avant** le versement.

1.3.4 Étude de l'existant : la plateforme OpenG2P

Avant de proposer une solution, il est essentiel de comprendre comment fonctionne la plateforme sur laquelle elle sera intégrée. Nous avons donc mené une étude approfondie d'OpenG2P, son architecture, ses modules, et ses processus métier, afin d'identifier précisément où se situent les faiblesses face à la fraude.

1.3.4.1 Présentation d'OpenG2P



FIGURE 1.5 – Logo de OpenG2P

OpenG2P est une plateforme open-source bâtie sur Odoo 17, conçue pour centraliser la gestion des programmes de protection sociale. Elle est déployée dans plusieurs pays (Kenya, Malawi, Sierra Leone) et couvre l'ensemble du cycle de vie d'un programme social, depuis l'enregistrement des bénéficiaires jusqu'au versement des aides. La plateforme s'organise autour de plusieurs modules interconnectés :

- **Registre social** : base de données centralisée des bénéficiaires avec leurs informations personnelles, démographiques et socio-économiques
- **Gestion des programmes** : définition des programmes d'aide, organisation des cycles de versement (trimestriels, mensuels)
- **Droits et paiements** : calcul automatisé des montants dus à chaque bénéficiaire et suivi de chaque versement
- **Données transactionnelles** : registre des coordonnées téléphoniques et bancaires utilisées pour les transferts
- **Groupes et ménages** : gestion des liens familiaux et des regroupements de bénéficiaires

1.3.4.2 Processus actuel dans OpenG2P et les limites face au risque de fraude

Le processus de contrôle anti-fraude dans OpenG2P repose actuellement sur un système minimal : quelques vérifications manuelles effectuées par les agents sociaux lors de l'enregistrement des bénéficiaires, complétées par des audits post-versement sur un échantillon limité. Aucun moteur automatisé de détection n'est intégré à la plateforme.

Afin de mieux comprendre les limites du système actuel, nous présentons ci-dessous une analyse des mécanismes de contrôle existants.

La Figure 1.6 illustre le processus actuel de vérification dans OpenG2P, où l'agent social consulte manuellement les fiches des bénéficiaires et effectue des contrôles ponctuels sans outil d'aide à la décision :

Les modules OpenG2P et la chaîne de distribution des aides sociales

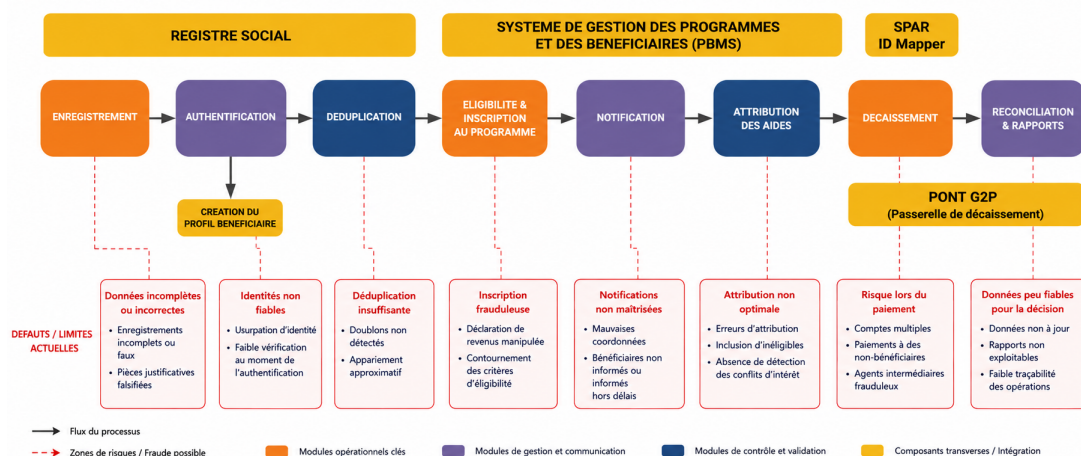


FIGURE 1.6 – Processus actuel dans OpenG2P et les zones de vulnérabilité à la fraude

Comme le montre cette figure, **aucun mécanisme de détection automatique** n'intervient entre l'enregistrement et le versement. La fraude peut s'infiltrer à chaque étape sans être détectée, et les contrôles manuels post-versement ne couvrent qu'une fraction des cas.

1.3.5 Objectif du projet

Face à ces constats, notre projet a pour ambition de transformer fondamentalement la manière dont les programmes de protection sociale luttent contre la fraude. L'objectif n'est pas simplement de détecter des cas suspects, mais de **changer le paradigme** : passer d'un système réactif, qui découvre la fraude après le versement, à un système intelligent qui la prévient avant qu'elle ne cause des dégâts. Pour y parvenir, nous avons structuré notre travail autour de cinq objectifs principaux :

- **Détection proactive** : identifier les bénéficiaires à risque avant le versement, au moment même de leur enregistrement dans le système.
- **Analyse multi-couches** : croiser plusieurs signaux complémentaires règles métier, apprentissage automatique, comportements inhabituels et liens entre bénéficiaires pour ne laisser aucun angle mort.
- **Explicabilité des décisions** : accompagner chaque alerte d'une explication claire en français, permettant à un agent social de comprendre et de justifier pourquoi un cas est signalé.
- **Amélioration continue** : créer un système qui apprend des retours des enquêteurs et devient plus performant au fil du temps.
- **Accessibilité et intégration** : offrir une solution 100% open-source, gratuite et intégrée directement à la plateforme OpenG2P que les gouvernements utilisent déjà.

1.4 Choix méthodologique et conceptuel

1.4.1 Méthodologie de conception

Pour la conception de ce projet, nous avons adopté le langage de modélisation **UML** afin de représenter les différentes vues du système de manière standardisée. Nous avons retenu quatre diagrammes : cas d'utilisation, classes, séquence et déploiement, qui permettent de couvrir respectivement les interactions, la structure, le flux de scoring et l'architecture Docker du système.

1.4.2 Méthodologie adoptée

La nature exploratoire d'un projet de data science — où les données évoluent, les modèles doivent être comparés et les hypothèses remises en question — impose une méthodologie flexible. Nous avons donc adopté la méthode **SCRUM**, un cadre agile structuré en sprints de 1 à 2 semaines, permettant de livrer des incréments fonctionnels et d'intégrer les retours d'EY Tunisia de manière continue.

1.4.2.1 Comparaison entre les méthodologies traditionnelles et agiles

Critère	Traditionnelle	Agile
Approche	Linéaire et séquentielle	Itérative et incrémentale
Exigences	Figées au départ	Évolutives
Changement	Difficile et coûteux	Flexible et intégré
Client	Impliqué au début et à la fin	Impliqué en continu
Livraison	Une seule fois à la fin	Fréquente (cycles courts)
Risques	Identifiés tardivement	Maîtrisés en continu

TABLEAU 1.2 – Méthodologies Traditionnelles vs Agiles

1.4.2.2 Pilotage du projet

Le projet a été organisé en six phases : cadrage, conception, génération de données, modélisation ML, intégration et tests. L'équipe SCRUM est composée comme suit :

Product Owner : EY Tunisia

Scrum Master : [Cyrine Garram, Mariem Abcha]

Équipe de développement : [Halim Trabelsi]

1.5 Diagramme de Gantt

Le diagramme de Gantt ci-dessous présente la planification temporelle du projet, découpée en phases et en sprints sur la durée du stage. La Figure ?? illustre cette répartition en utilisant les mois comme unité de temps.

FIGURE 1.7 – Diagramme de Gantt du projet

Conclusion

Ce chapitre a posé les fondations du projet : après avoir présenté le cadre institutionnel, étudié les lacunes d'OpenG2P et du marché, et défini nos objectifs, nous avons adopté une méthodologie SCRUM adaptée à la nature itérative d'un projet de data science. Le chapitre suivant sera consacré à **l'état de l'art**, où nous examinerons les techniques de détection de fraude dans la littérature scientifique afin de justifier les choix techniques de notre solution.

Chapitre 2

Etat de l'art

Introduction

Avant de détailler la conception et la réalisation de notre solution, il est essentiel de poser les bases théoriques sur lesquelles elle repose. Ce chapitre a pour objectif de présenter les notions fondamentales nécessaires à la compréhension du projet, et d'examiner les techniques existantes dans la littérature scientifique pour la détection de fraude.

2.1 Définitions et concepts clés

Il est indispensable de définir le vocabulaire et les notions fondamentales qui seront mobilisés tout au long de ce rapport. Cette section a pour but de rendre le reste du document accessible à tout lecteur, y compris ceux qui ne sont pas familiers avec les domaines de l'intelligence artificielle ou de l'analyse de données.

2.1.1 Concepts liés à la fraude et à la protection sociale :

- **Programme G2P (Government-to-Person)** : mécanisme par lequel un gouvernement verse des aides financières directement aux citoyens éligibles (allocations familiales, bourses, aides alimentaires). C'est le contexte métier de notre projet.
- **Détection de fraude** : ensemble des méthodes permettant d'identifier des comportements malhonnêtes ou irréguliers dans un système. Dans notre cas, il s'agit de repérer les bénéficiaires qui tentent de recevoir indûment des aides sociales.
- **Vrai positif** : cas où le système identifie correctement une transaction ou un bénéficiaire réellement frauduleux comme étant frauduleux.
- **Vrai négatif** : cas où le système identifie correctement une transaction ou un bénéficiaire légitime comme étant non frauduleux.
- **Faux positif** : cas où le système signale un bénéficiaire comme fraudeur alors qu'il est légitime. Un excès de faux positifs surcharge inutilement les enquêteurs et nuit à la confiance envers le système.
- **Faux négatif** : cas où le système laisse passer un vrai fraudeur sans le détecter. C'est la situation la plus coûteuse, car l'argent est versé à une personne qui n'y a pas droit.
- **Collusion** : forme de fraude collective où plusieurs individus collaborent pour tromper le système — par exemple, en partageant un même compte bancaire ou un même numéro de téléphone pour multiplier les inscriptions.

2.1.2 Concepts techniques : Intelligence artificielle et données

L'intelligence artificielle, et plus précisément le machine learning, est au cœur de notre solution. Pour comprendre comment notre système apprend à distinguer un bénéficiaire légitime d'un fraudeur, il est nécessaire de maîtriser les notions suivantes :

- **Machine Learning (apprentissage automatique)** : branche de l'intelligence artificielle où un programme apprend à prendre des décisions à partir d'exemples passés, sans être explicitement programmé pour chaque cas. Le programme identifie des régularités

dans les données et les utilise pour faire des prédictions sur de nouveaux cas.

- **Apprentissage supervisé** : type de machine learning où le modèle apprend à partir d'exemples dont on connaît déjà la réponse (par exemple : « ce bénéficiaire est un fraudeur » ou « ce bénéficiaire est légitime »). Le modèle apprend la relation entre les caractéristiques d'un bénéficiaire et son statut, puis applique cette connaissance aux nouveaux cas.
- **Apprentissage non supervisé** : type de machine learning où le modèle ne dispose pas de réponses connues. Il cherche à identifier des comportements atypiques ou des groupes naturels dans les données, sans savoir à l'avance ce qui est normal ou anormal.
- **Feature engineering (extraction de caractéristiques)** : étape qui consiste à transformer les données brutes en variables exploitables par un modèle. Par exemple, à partir du nombre de téléphones partagés et du nombre de comptes bancaires partagés, on calcule un « score de risque réseau » qui résume l'information en un seul chiffre.
- **Data leakage (fuite de données)** : erreur dans la construction du modèle où des informations liées à la réponse (le label fraude/légitime) se retrouvent accidentellement dans les données d'entraînement. Le modèle semble alors parfait, mais ses performances sont artificiellement gonflées et ne se reproduisent pas en conditions réelles.
- **AUC-ROC (Area Under the Receiver Operating Characteristic Curve)** : métrique d'évaluation qui mesure la capacité d'un modèle à distinguer les fraudeurs des légitimes, sur une échelle de 0 à 1. Un AUC de 0.5 signifie que le modèle ne fait pas mieux que le hasard ; un AUC de 1.0 signifie une détection parfaite. Dans la pratique, un AUC supérieur à 0.85 est considéré comme bon.
- **Accuracy (exactitude)** : proportion de prédictions correctes (vrais positifs + vrais négatifs) parmi l'ensemble des prédictions effectuées.
- **Précision (precision)** : proportion de cas réellement frauduleux parmi tous les cas que le système a prédits comme frauduleux.
- **Rappel (recall)** : proportion de cas réellement frauduleux que le système a effectivement réussi à détecter parmi tous les cas frauduleux existants.
- **F1-score** : moyenne harmonique de la précision et du rappel, offrant une mesure équilibrée de la performance du modèle, surtout lorsque les classes sont déséquilibrées (comme c'est souvent le cas en détection de fraude).

2.1.3 Explicabilité et génération d'explications

Un système de détection de fraude ne peut être utile que s'il est compréhensible par ses utilisateurs. Un agent social doit pouvoir expliquer à un bénéficiaire pourquoi son dossier a été signalé. C'est le rôle de l'explicabilité :

- **XAI (Explainable Artificial Intelligence)** : domaine de l'intelligence artificielle qui vise à rendre les décisions des modèles compréhensibles par des humains. Plutôt qu'une « boîte noire » qui dit simplement « fraude détectée », un système explicable indique *pourquoi* : quels facteurs ont pesé dans la décision.
- **SHAP (SHapley Additive exPlanations)** : méthode d'explicabilité qui attribue à chaque caractéristique du bénéficiaire une contribution précise au score final. Par

exemple : « le partage de téléphone avec 4 autres bénéficiaires a augmenté le score de risque de +0.15 ». SHAP est fondé sur la théorie des jeux (valeurs de Shapley) et constitue aujourd'hui le standard de l'industrie.

- **LLM (Large Language Model)** : modèle d'intelligence artificielle capable de comprendre et de générer du texte en langage naturel. Dans notre projet, nous utilisons Mistral (via Ollama) pour transformer les facteurs SHAP et les drapeaux de règles en une **explication rédigée en français**, compréhensible par un agent social non-technique.
- **RAG (Retrieval-Augmented Generation)** : technique qui enrichit la réponse d'un LLM en lui fournissant des documents pertinents avant la génération. Dans notre contexte, le LLM reçoit les facteurs SHAP, les règles déclenchées et le score du bénéficiaire comme contexte, ce qui lui permet de produire une explication précise et fondée sur des faits réels, et non sur des connaissances génériques.

2.1.4 Analyse de graphe et détection de réseaux

La fraude n'est pas toujours l'acte d'un individu isolé. Souvent, plusieurs personnes collaborent en partageant des ressources (téléphones, comptes bancaires). L'analyse de graphe permet de révéler ces réseaux invisibles :

- **Graphe** : structure mathématique composée de **nœuds** (les entités — ici, les bénéficiaires) et d'**arêtes** (les relations entre eux — ici, le partage d'un téléphone ou d'un compte bancaire). Si deux bénéficiaires partagent le même numéro de téléphone, une arête les relie.
- **Détection de communautés** : technique qui identifie des groupes de nœuds fortement connectés entre eux. Une communauté dense de 5 à 15 bénéficiaires partageant plusieurs téléphones et comptes bancaires est un signal fort de fraude organisée.
- **Centralité (PageRank)** : mesure d'importance d'un nœud dans un réseau. Un bénéficiaire très connecté (lié à de nombreux autres bénéficiaires suspects) obtient un score de centralité élevé. Notre solution utilise l'algorithme PageRank, le même que celui inventé par Google pour classer les pages web adapté au contexte de la fraude sociale.

Ces définitions posées, nous disposons maintenant du vocabulaire nécessaire pour examiner en détail chacune des techniques de détection de fraude dans les sections suivantes.

2.1.5 Identification des acteurs

Notre système de détection de fraude interagit avec plusieurs acteurs, qu'ils soient des utilisateurs humains ou des systèmes informatiques. L'identification de ces acteurs est une étape essentielle pour définir les besoins fonctionnels de la solution. Le Tableau ?? présente les acteurs internes qui utilisent directement le système, tandis que le Tableau ?? décrit les acteurs externes et les systèmes avec lesquels notre solution communique.

Acteur	Rôle
Bénéficiaire	Personne inscrite au programme de protection sociale qui reçoit (ou tente de recevoir) les aides. C'est l'entité analysée par le moteur de détection de fraude.
Agent social	Personnel chargé de l'enregistrement des bénéficiaires dans OpenG2P et de la vérification des informations sur le terrain. Il est le premier point de contact avec le bénéficiaire.
Analyste fraude	Utilisateur qui consulte les alertes générées par le système, examine les explications fournies (SHAP + LLM) et décide des actions à entreprendre : validation, investigation ou blocage du versement.
Administrateur	Responsable de la configuration des règles métier, des seuils de détection et de la gestion technique de la plateforme (utilisateurs, droits d'accès, paramètres ML).
Superviseur	Responsable qui consulte les tableaux de bord pour suivre les indicateurs globaux (taux de fraude, performance du modèle, tendances), du réentraînement des modèles et prendre des décisions stratégiques sur le programme.

TABLEAU 2.1 – Acteurs internes du système

Acteur / Système	Rôle
OpenG2P / Odoo	Plateforme source qui gère le registre des bénéficiaires, les programmes et les paiements. Elle transmet les données au moteur de détection et reçoit les alertes de fraude en retour.
PIAF (moteur de fraude)	Système central qui reçoit les données, applique les quatre couches d'analyse (règles, ML, graphe, anomalie), produit un score de risque et génère une explication en langage naturel.
Organisme de protection sociale	Entité institutionnelle (ministère, agence gouvernementale) bénéficiaire des rapports de fraude, responsable des politiques publiques et des décisions budgétaires.

TABLEAU 2.2 – Acteurs externes et systèmes

2.1.6 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions concrètes que chaque acteur peut réaliser dans le système. Ils ont été identifiés à partir de l'étude de l'existant et des objectifs définis au chapitre précédent. Le Tableau ?? à ?? détaillent ces besoins par acteur.

ID	Besoin fonctionnel	Description
BF01	Enregistrer un bénéficiaire	Saisir les informations personnelles, démographiques et bancaires d'un nouveau bénéficiaire dans le registre OpenG2P
BF02	Consulter la fiche bénéficiaire	Accéder aux données complètes d'un bénéficiaire (identité, ménage, programmes, historique de paiements)
BF03	Déclencher un scan unitaire	Lancer manuellement l'analyse de fraude sur un bénéficiaire donné et consulter le score de risque obtenu

TABLEAU 2.3 – Besoins fonctionnels **Agent social**

ID	Besoin fonctionnel	Description
BF04	Consulter les alertes de fraude	Visualiser la liste des cas signalés avec leur score, niveau de risque et règles déclenchées
BF05	Examiner les explications	Consulter les facteurs SHAP et l'explication en langage naturel générée par le LLM pour chaque cas signalé
BF06	Donner un feedback	Valider ou infirmer une alerte (confirmer fraude, faux positif, indéterminé) afin d'alimenter la boucle d'amélioration continue
BF07	Suivre l'investigation	Gérer le workflow d'investigation via le tableau Kanban dans Odoo (assignation, notes, changement de statut)
BF08	Exporter un rapport	Générer un rapport d'investigation au format PDF ou CSV pour un cas donné

TABLEAU 2.4 – Besoins fonctionnels **Analyste fraude / Enquêteur**

ID	Besoin fonctionnel	Description
BF09	Gérer les règles métier	Ajouter, modifier, activer ou désactiver les règles de détection de fraude (seuils, poids, conditions)
BF10	Recharger les règles	Appliquer les modifications des règles en temps réel sans redémarrer le système
BF11	Réentraîner le modèle	Déclencher un réentraînement du modèle ML avec les nouvelles données et les feedbacks collectés
BF12	Gérer les utilisateurs	Créer, modifier ou supprimer les comptes utilisateurs et leurs droits d'accès

TABLEAU 2.5 – Besoins fonctionnels **Administrateur**

ID	Besoin fonctionnel	Description
BF13	Consulter le tableau de bord	Visualiser les indicateurs globaux : nombre de cas, répartition par niveau de risque, tendances temporelles, taux de faux positifs
BF14	Consulter les hotspots géographiques	Afficher une carte de chaleur des zones à forte concentration de fraude détectée
BF15	Suivre la performance du modèle	Consulter les métriques ML (AUC, précision, rappel) et détecter une éventuelle dégradation
BF16	Versionner les modèles	Consulter l'historique des modèles dans ML-flow, comparer les performances et effectuer un rollback si nécessaire

TABLEAU 2.6 – Besoins fonctionnels **Superviseur / Décideur**

ID	Besoin fonctionnel	Description
BF17	Scanner en masse	Analyser automatiquement l'ensemble des bénéficiaires d'un programme en un seul appel (scoring batch)
BF18	Scoring temps réel	Déclencher automatiquement l'analyse de fraude à chaque nouvel enregistrement de bénéficiaire via webhook
BF19	Calculer le score hybride	Combiner les quatre sous-scores (règles, ML, anomalie, graphe) en un score final unique via le pipeline
BF20	Générer l'explication	Produire automatiquement les facteurs SHAP et une explication en français via le LLM pour chaque cas scoré
BF21	Synchroniser avec Odoo	Créer automatiquement les cas de fraude dans le module Odoo et mettre à jour leur statut
BF22	Envoyer des alertes	Notifier les analystes lorsqu'un cas CRITICAL ou HIGH est détecté

TABLEAU 2.7 – Besoins fonctionnels **Système (PIAF / OpenG2P)**

2.1.7 Identification des besoins non fonctionnels

Les besoins non fonctionnels concernent les contraintes à prendre en considération pour mettre en place une solution adéquate aux attentes des clients. Notre plateforme doit nécessairement assurer ces besoins :

- **Sécurité** : La plateforme doit garantir la confidentialité et l'intégrité des données personnelles des bénéficiaires, en conformité avec les réglementations en vigueur (ex : RGPD)
- **Scalabilité** : La solution doit être capable de gérer un nombre croissant de bénéficiaires et de transactions sans dégradation des performances.

- **Accessibilité** : L'interface utilisateur doit être intuitive et accessible à des agents sociaux ayant des compétences techniques limitées.
- **Interopérabilité** : Le moteur de détection doit s'intégrer facilement avec d'autres systèmes utilisés par les organismes de protection sociale.
- **Maintenabilité** : Le code doit être bien structuré, documenté et modulaire pour faciliter les mises à jour et les évolutions futures.

2.1.8 Diagramme de cas d'utilisation général

Le diagramme de cas d'utilisation est le meilleur outils pour décrire les interactions entre nos acteurs et notre système étudié. Il représente les différents cas d'utilisation du système, en décrivant les scénarios dans lesquels les acteurs interagissent avec celui-ci :

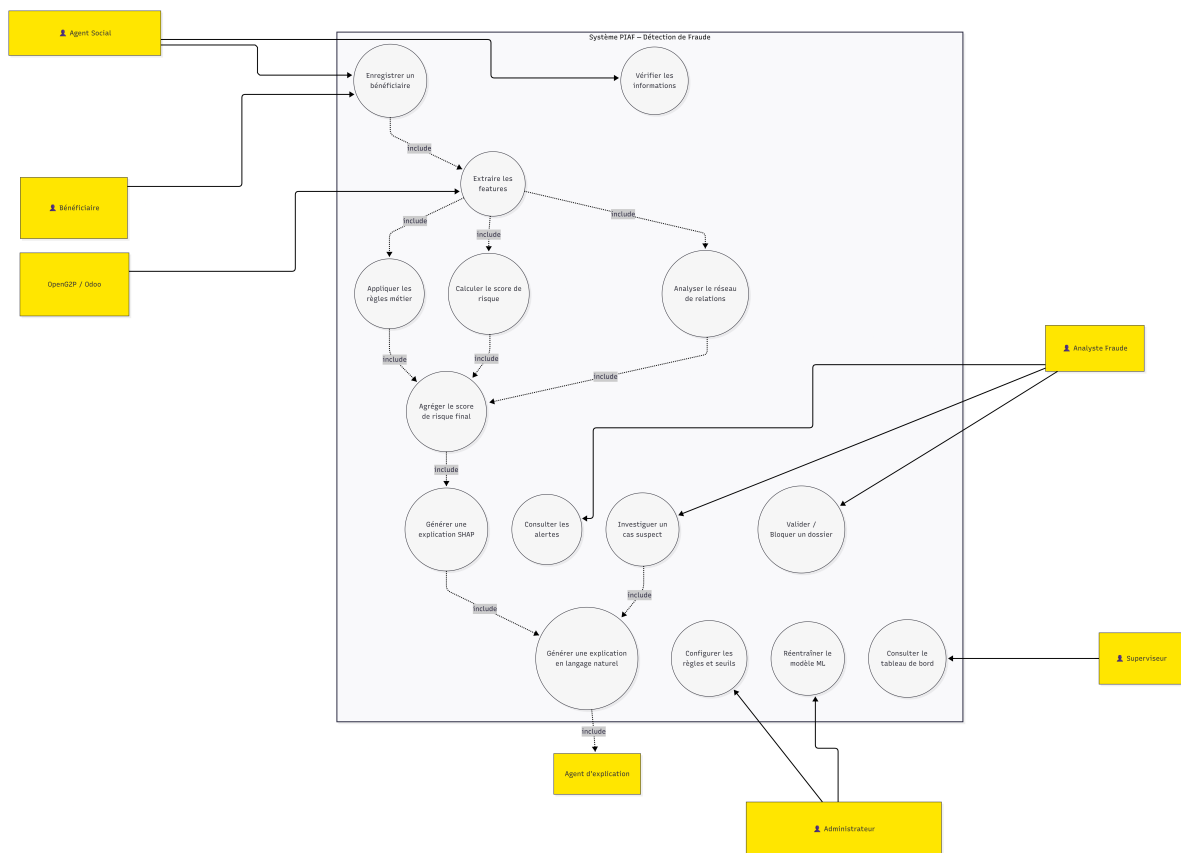


FIGURE 2.1 – Diagramme de cas d'utilisation général

2.1.9 Diagramme de classes général

Le diagramme de classes est l'un des outils de modélisation les plus efficaces pour représenter la structure statique d'un système. Il permet de décrire les différentes classes du système révélant leurs attributs définissant aussi les relations entre elles : l'héritage, l'association, l'aggrégation ou la composition. Il procure une vue d'ensemble de l'agencement et de l'organisation interne du système étudié. :

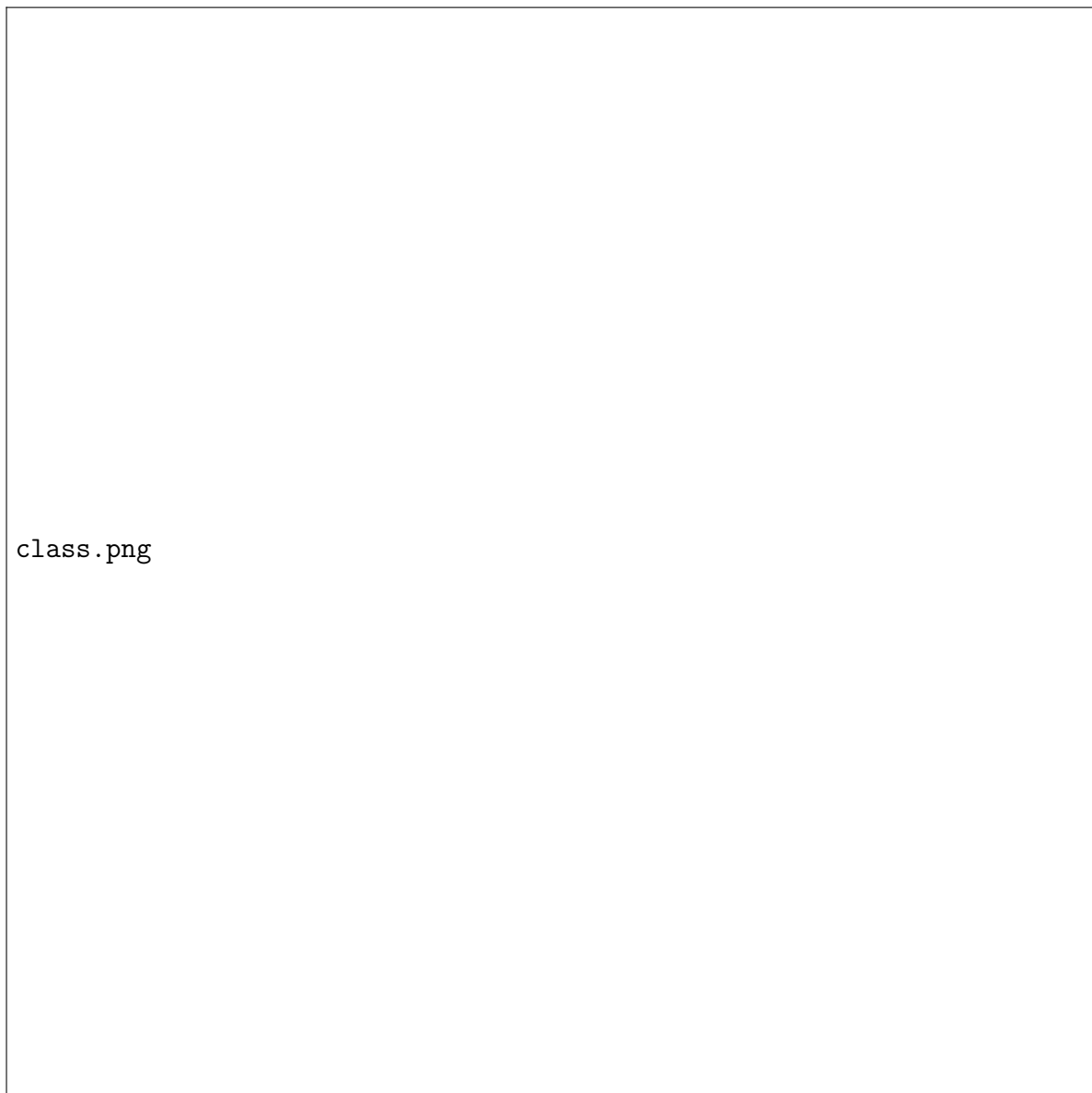


FIGURE 2.2 – Diagramme de classes général

2.2 Architecture physique

Cette section présente l'architecture matérielle et logicielle de l'application, en décrivant la mise en œuvre concrète des différents nœuds qui composent le système. La configuration physique, telle qu'illustrée sur le schéma, établit une distinction claire entre le poste de travail de l'utilisateur, les serveurs d'application, le serveur de persistance et les services distants.

Le système s'articule autour des composants physiques suivants :

2.2.1 La couche client (poste de travail)

Le client représente le terminal physique (ordinateur de bureau ou mobile) à partir duquel l'utilisateur final interagit avec la plateforme.

Navigateur Web : Il constitue l'interface logicielle (telle que Chrome ou Edge) installée sur la machine de l'utilisateur. Ce navigateur joue le rôle de point d'entrée unique, émettant

des requêtes HTTP vers le serveur Frontend et assurant le rendu visuel des pages web reçues.

2.2.2 Le serveur de présentation (frontend React)

Ce nœud applicatif est responsable de l'hébergement de l'interface utilisateur développée en React.js.

Communication : Ce serveur écoute les requêtes entrantes. Il capte les demandes provenant du navigateur et transmet les appels de données métier vers le serveur Backend.

2.2.3 Le serveur d'applications (Backend Spring Boot)

Au cœur de l'infrastructure physique se trouve le serveur backend, qui héberge la logique métier et la couche d'intégration. Comme l'illustre le schéma, ce serveur comprend plusieurs composants logiciels fondamentaux :

Noyau du framework : basé sur Spring Boot, il gère le cycle de vie de l'application et assure l'inversion de contrôle.

Persistance des données (JPA/Hibernate) : ce module, intégré au serveur, gère la connexion et les transactions avec la base de données relationnelle MySQL.

Intégration vectorielle (Vector DB) : le serveur backend intègre la logique nécessaire pour interagir avec la base de données vectorielle, facilitant ainsi les requêtes sémantiques et le mécanisme RAG (Retrieval-Augmented Generation).

Communication : Il expose les API REST sur le Port 8080 et orchestre les échanges vers les bases de données internes et les services tiers.

2.2.4 Le serveur de données (MySQL)

Il s'agit d'un serveur physique (ou virtuel) dédié au stockage persistant des données.

Rôle : il stocke toutes les informations structurées de l'application (utilisateurs, applications, résultats de tests) et communique avec le serveur backend.

2.2.5 Services externes

API LLM : service d'intelligence artificielle accessible via Internet, sollicité par le serveur backend pour la génération des quizzes et des réponses.

Serveur SMTP : serveur de messagerie utilisé pour l'envoi automatique d'e-mails de notification (confirmations d'inscription, résultats de tests).

La Figure 2.3 illustre l'architecture physique de notre application :

FIGURE 2.3 – Architecture physique

2.3 Architecture logique

Dans cette section, nous allons présenter les technologies, les frameworks et les composants matériels que nous avons choisis pour notre application. Notre application est basée sur une architecture client–serveur distribuée composée de plusieurs machines logiques : client, serveur d'application, serveur d'intelligence artificielle et serveur de base de données.

Le système est constitué des éléments suivants :

2.3.1 Client (React)

Le client, qui représente l'utilisateur accédant à l'application via un navigateur web, utilise une interface utilisateur développée en React.js, permettant aux acteurs du système (Candidat, Agent RH, Administrateur) d'interagir avec la plateforme.

Cette partie frontend, visible et interactive, repose sur React.js, une bibliothèque JavaScript favorisant des interfaces dynamiques et réactives via une architecture modulaire basée sur des composants réutilisables, chacun gérant son propre état et communiquant avec les autres par des propriétés (props) ou services.

L'application communique avec le serveur backend via des API REST sécurisées pour envoyer et récupérer les données de manière fluide.

2.3.2 Serveur Backend (Spring Boot)

Pour la partie back-end, nous avons adopté l'architecture MVC qui se compose de trois parties principales : le modèle, la vue et le contrôleur. L'architecture MVC permet de séparer les responsabilités de chaque composant, ce qui facilite la maintenance, l'évolution et la réutilisation du code. Cette architecture est utilisée avec le framework Spring Boot, qui permet de développer des applications robustes et sécurisées basées sur des API REST.

Modèle :

Le modèle représente la partie responsable de la gestion des données et de la logique métier. Il interagit avec la base de données à l'aide d'un ORM tel que JPA/Hibernate.

Vue :

Dans notre architecture, la vue correspond à l'interface utilisateur développée en Frontend. La vue communique avec le backend via des API REST sécurisées.

Contrôleur :

Le contrôleur assure la communication entre la vue et le modèle. Il reçoit les requêtes envoyées par le client, traite les données, appelle les services nécessaires et retourne les résultats. Il est responsable de la gestion des requêtes HTTP et de l'orchestration des opérations de l'application.

2.3.2.1 Couche de sécurité de MVC

La couche de sécurité de notre application utilise Spring Security avec JWT afin de sécuriser l'accès aux API. Lorsqu'un utilisateur se connecte, un token JWT est généré et envoyé au client. Ce token est utilisé pour vérifier l'identité de l'utilisateur lors de chaque requête vers le serveur. Cette méthode garantit la confidentialité, l'intégrité et l'authenticité des données échangées.

Le système inclut également :

- Vérification par email avec code.
- Gestion des rôles (Admin, RH, Candidat).
- Protection des API.
- Authentification sécurisée.

Le backend est développé avec Spring Boot et expose des API REST sécurisées.

2.3.3 Serveur Intelligence Artificielle

Un serveur dédié à l'intelligence artificielle est utilisé pour traiter les fonctionnalités avancées du système, telles que :

- Analyse automatique des CV à l'aide des techniques de traitement du langage naturel (NLP).
- Génération automatique de quiz.
- Reconnaissance faciale pour l'authentification et la vérification de l'identité des candidats.
- Organisation et planification intelligente des entretiens.
- Chatbot intelligent basé sur l'architecture RAG pour assister les recruteurs.

Ce serveur communique avec le backend via des services API.

2.3.4 Serveur Base de données

La base de données est utilisée pour stocker toutes les informations de l'application :

- utilisateurs.
- candidatures.
- Profil et CV.
- résultats des tests.
- notes des entretiens.
- scores IA.

Une base de données relationnelle "**MySQL**" est utilisée pour assurer la cohérence et la sécurité des données.

2.3.5 Serveur DevOps / Déploiement

Une couche DevOps est utilisée pour automatiser :

- le déploiement.
- les tests.
- l'intégration continue.
- la surveillance.

La Figure 2.4 illustre l'architecture logique de notre application :

FIGURE 2.4 – Architecture logique

2.4 Environnement logiciel

Dans ce qui suit, nous présentons les outils logiciels utilisés lors de développement de notre plateforme.

2.4.1 Outils de conception

* **Visual Paradigm** : Visual Paradigm[7] est un logiciel de modélisation visuelle et de conception utilisé pour créer des modèles et des diagrammes dans le cadre du développement des logiciels et de la gestion des projets. Il offre une large gamme de fonctionnalités pour visualiser, analyser et documenter des systèmes logiciels complexes.

FIGURE 2.5 – Visual paradigm logo

* **LATEX** : Le LATEX [?] est un langage de composition de documents utilisé pour la création de rapports, d'articles scientifiques, de thèses, de livres et d'autres types de documents. Il est particulièrement apprécié dans les domaines académiques et scientifiques en raison de sa capacité à générer des documents de haute qualité typographique avec une mise en page professionnelle.

FIGURE 2.6 – LATEX logo

2.4.2 Outils de développement

Logo	Technologie
	Visual Studio Code est un éditeur de code léger et extensible créé par Microsoft, offrant des fonctionnalités avancées telles que le débogage intégré, l'autocomplétion intelligente, la refactorisation du code et une intégration native de Git.[1]
	MySQL est un système de gestion de bases de données relationnelles open source très répandu, permettant de stocker, organiser et manipuler des données structurées à travers le langage SQL.[5]
	Spring Boot est un framework Java open source dérivé du framework Spring, conçu pour simplifier le développement d'applications autonomes grâce à un ensemble de bibliothèques préconfigurées qui accélèrent la mise en place et la gestion des projets.
	ReactJS est une bibliothèque JavaScript dédiée à la construction d'interfaces utilisateur interactives côté client. Développée par Facebook en 2013, elle est aujourd'hui maintenue par Meta et une large communauté open source.[2]
	Postman est un outil largement adopté par les développeurs pour tester, concevoir et documenter des API. Il propose une interface graphique intuitive permettant d'envoyer des requêtes HTTP, de configurer des paramètres et d'analyser les réponses obtenues..[9]
	Trello est une application web de gestion de projet collaborative reposant sur un système de tableaux permettant aux équipes d'organiser et de gérer leurs tâches et projets en temps réel.[10]
	GitHub est une plateforme cloud dédiée au développement logiciel collaboratif, offrant aux développeurs la possibilité d'héberger, de partager et de contribuer à des projets de code source à l'aide du système de versionnage Git.[11]
	Docker est une plateforme logicielle permettant de créer, déployer et exécuter des applications dans des conteneurs isolés et portables, garantissant ainsi une cohérence d'environnement entre les différentes phases de développement.[11]
	Jenkins est un serveur d'automatisation open source utilisé pour orchestrer les processus de build, de test et de déploiement des applications logicielles, facilitant ainsi la mise en œuvre de l'intégration continue et de la livraison continue.

TABLEAU 2.8 – Outils logiciels

2.5 Conclusion

Dans ce chapitre, nous avons structuré notre travail en identifiant les besoins fonctionnels et non fonctionnels de notre application, ainsi que les rôles des utilisateurs impliqués. Nous avons également présenté le backlog de notre système et fourni une planification détaillée des sprints à réaliser. Enfin, nous avons sélectionné l'architecture de notre projet et établi l'environnement de travail approprié dans le but de transformer notre vision en réalité fonctionnelle.